CG2271 Real Time Operating Systems

2025/26 Semester 2

Final Report

B01_04	
Name	Matric Number
Pran Tanprasertkul	A0300807N
Abe Foo Qian Yin	A0306861A
Emry Daniel Bin Abdul Lathiff	A0309131R
Chua Jia En	A0282463B

Problem statement

Residents face the issue of their laundry getting wet due to unpredictable rain or high humidity. Our IoT-enabled smart laundry monitoring system tracks real-time environmental data to provide instant local and remote alerts.

Sensors and Actuators used

Sensor/Actuator	Description of use	Libraries used (if any)
Water level sensor	Detects presence of rainwater. Polls to determine if water level exceeds threshold to trigger an alert.	-
Photoresistor	Monitors ambient brightness to evaluate if it is sunny enough for clothes to dry. If light values indicate darkness, it is considered an unfavorable environment.	-
Temperature & humidity sensor	Tracks the ambient temperature and humidity levels to evaluate if the air is too damp or cold for drying laundry efficiently.	DHT.h (Adafruit Unified Sensor Library)
Joystick	Primary physical input for user interface. Allows adjusting of sensor sensitivity, toggle buzzer/LED.	-
LCD	Provides real-time local feedback by displaying the current settings and the mode of the operation once the joystick is moved.	LiquidCrystal_I2C.h

Buzzer (Active)	Primary auditory actuator. Beeps upon unfavorable environmental conditions.	-
LED (MCXC444)	Flashes red and green to visually warn when alert packet is received over UART from the ESP32.	-
Telegram Bot	Shows updates to user, allows for management of settings without joystick.	UniversalTelegramBot.h

Hardware Architecture

The circuit diagrams below show the connections between components and devices.

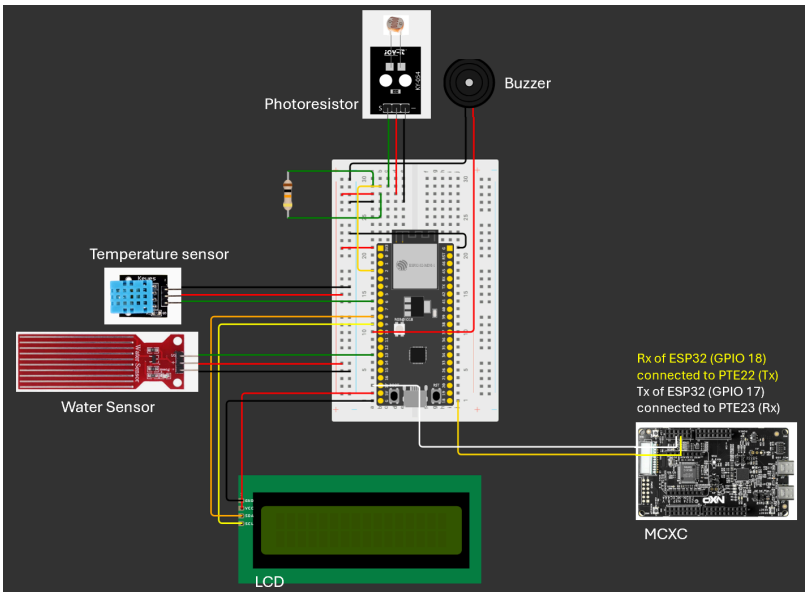

Fig 1: Circuit diagram of ESP32

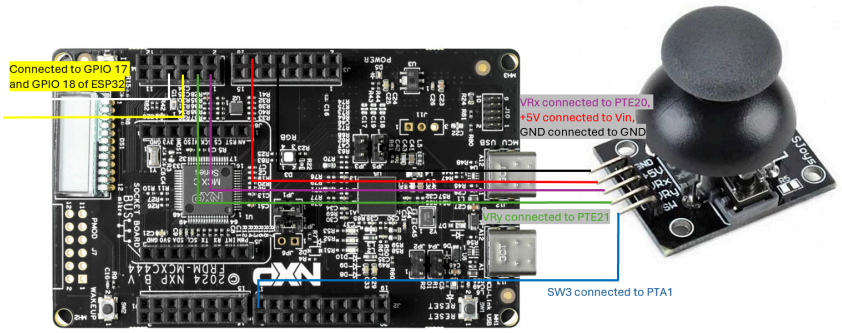

Fig 2: Circuit diagram of MCXC444

Software Architecture

State Management

The system manages states by comparing real-time sensor data against set thresholds. Sensors on the ESP32 measure environmental variables through polling and evaluate them against the previously defined limits. For instance, the water level measured by the water sensor is compared using the variable **waterPercentage** against **waterThresholdPercent**. Similar logic is used for brightness, temperature and humidity. For temperature, the variable **currentTemp** gets compared against **tempThreshold**. When a limit is exceeded, the global boolean flag **envAlertActive** changes from false to true, which in turn causes alarms to be triggered with the help of other flags like **alarmTriggered**. In addition, the **systemActive** flag keeps track of whether the monitoring system is currently active or is paused.

```
float currentTemp = 0.0;
float currentHum = 0.0;
int lightSensorValue = 2000;
int waterPercentage = 0;
bool rainDetected = false;

// Configurable Thresholds
float tempThreshold = 30.0;
float humThreshold = 85.0;
int lightThreshold = 4095;     // Maps from currentLightPercent

const int waterThresholdPercent = 70;
const int dryValue = 0;
const int wetValue = 3000;

unsigned long lastTelegramPoll = 0;
const unsigned long pollDelay = 1000;
unsigned long lastSensorRead = 0;
const unsigned long sensorDelay = 2000;
unsigned long lastAlertTime = 0;
unsigned long alertCooldown = 15000;

bool systemActive = true;
bool alarmTriggered = false;
bool envAlertActive = false;
unsigned long buzzerTimer = 0;
bool buzzerActive = false;

bool alertWithBuzzer = true;
bool alertWithLED = true;
bool alertWithTele = true;
```

Fig 3: Environment variables and global boolean flags

On the MCXC444, user input state is managed through enumeration of JoyState_t. Edge detection is implemented by comparing current_state against prev_state. This prevents continuous commands from being sent in the event a user holds the joystick in a particular direction for a longer duration.

When **requestToggleLED** or **requestToggleBuzzer** is true in **SendData_t**, the respective boolean variables are toggled. A request-based structure is sent instead of a change-based structure, where the new setting is sent from MCXC directly to override the current settings, and is done to ensure data accuracy as the settings can change due to Telegram too.

```c
typedef struct __attribute__((packed)) {
    uint8_t startByte; // 0xAA
    bool requestIncreaseLightSensitivty;
    bool requestIncreaseHumiditySensitivity;
    bool requestToggleLED;
    bool requestToggleBuzzer;
    bool requestDisplaySwitch;
} SendData_t;
```

```c
bool alertWithBuzzer = true;
bool alertWithLED = true;
bool alertWithTele = false;
```

Fig 4: Alert settings

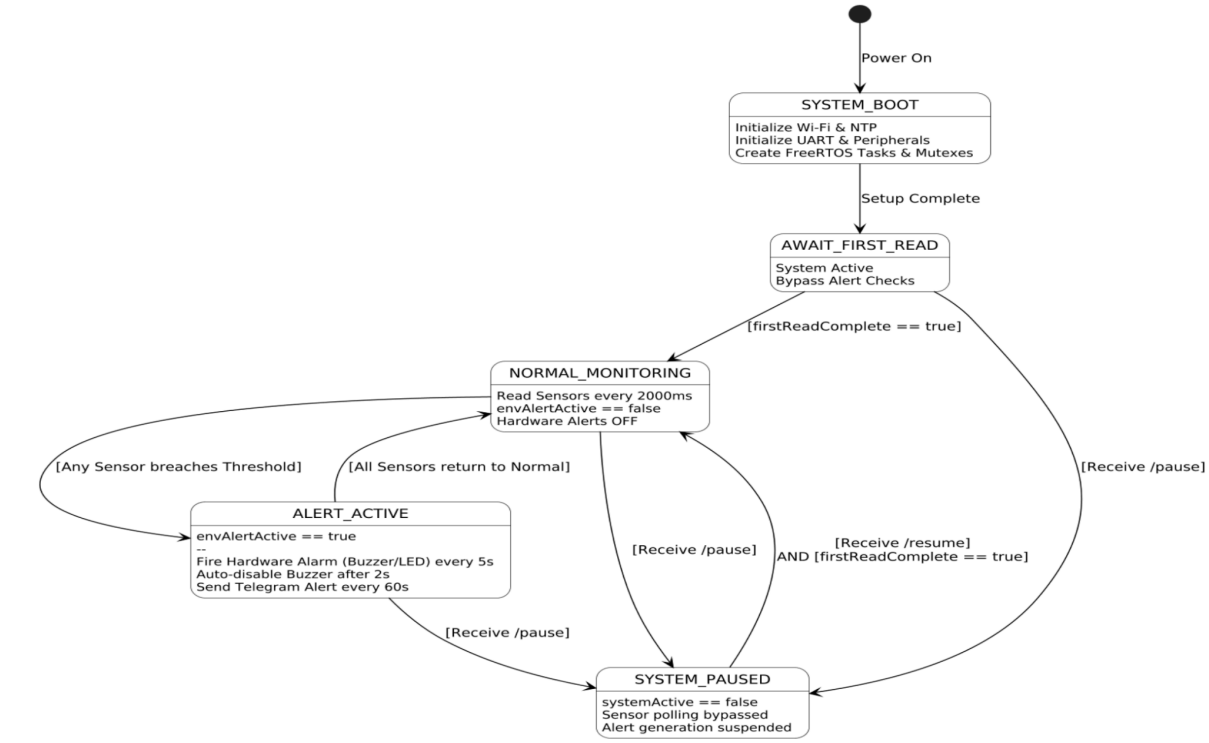

Fig 5: FSM Diagram

Task Structure

The workload is distributed across both the ESP32 and MCXC444 microcontrollers to ensure real-time responsiveness of the system.

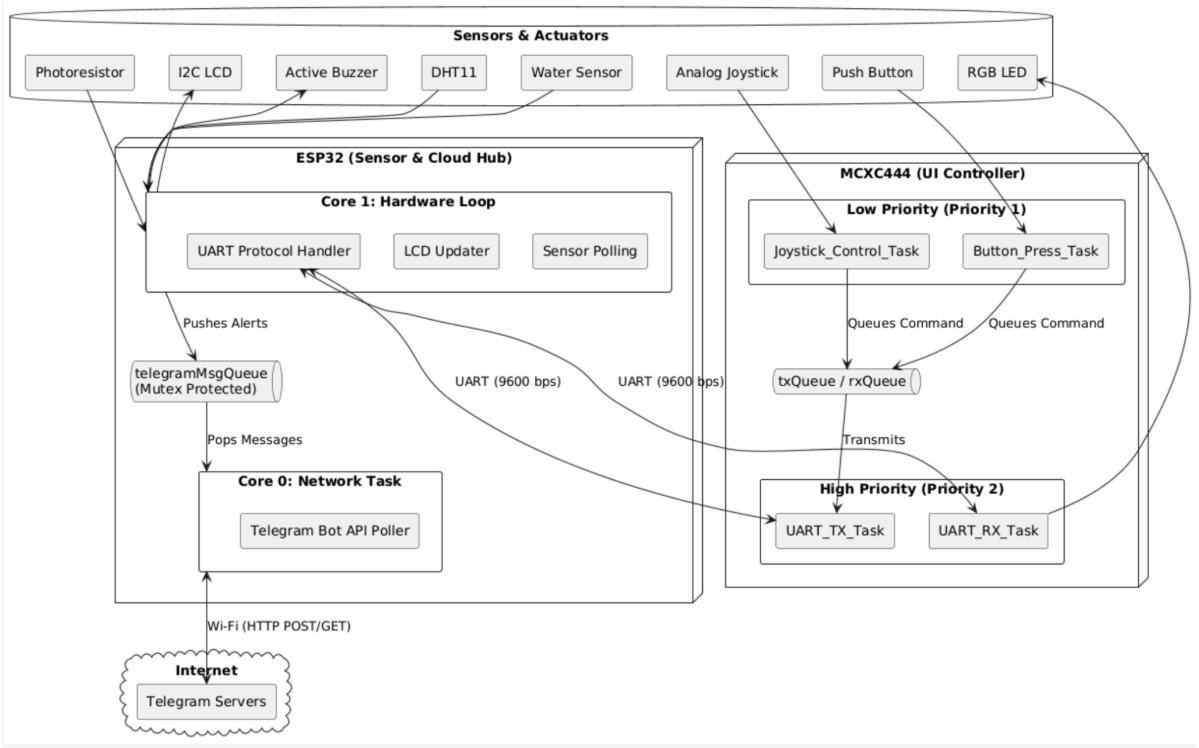

Fig 6: Software Architecture Diagram

Firstly, we have the ESP32 managing tasks directly related to our sensors, which are constantly polling to monitor surroundings. On top of that, it also handles UART communication with the MCX444 and tells the LCD what to display, as well as manage Wi-Fi network requests while being linked to the Telegram API when there are user inputs.

Secondly, The MCXC444 serves as the system's remote user interface. It runs five distinct FreeRTOS tasks. The tasks are assigned specific priority levels (where a higher number indicates a higher priority) to ensure that critical inter-board communication preempts standard user inputs.

Task	Priority	Description
UART_RX_TASK	2 (High)	Constantly listens for incoming data packets from the ESP32. When an alert packet is received, it immediately processes the data to trigger flashing of the onboard RGB LED.
UART_TX_TASK	2 (High)	Waits for data in the transmission queue and sends user configuration changes to the ESP32 (e.g. adjusting sensitivity thresholds).
Joystick_Control_Task	1 (Low)	Periodically polls the ADC to detect X and Y

		joystick movements, allowing the user to remotely adjust humidity and light sensitivities or change the buzzer volume.
Button_Press_Task	1 (Low)	Remains blocked state till awakened by a hardware interrupt. When joystick button is pressed, ESP32 changes the LCD display.
xQueue_Print_Task	1 (Low)	Receives string messages from other tasks via a queue and prints them one by one, preventing output from becoming garbled when multiple tasks attempt to log data simultaneously.

The High (Priority 2) tasks manage the UART communication between the ESP32 and MCXC444. Their assigned higher priority is to ensure that incoming and outgoing data packets are processed immediately without dropping bytes. The Low (Priority 1) tasks handle local user inputs and are for logging purposes, which will not require the CPU to the same extent.

Coordination & Synchronisation

To maximize CPU efficiency and ensure real-time responsiveness, the MCXC444 avoids resource-heavy polling which has to constantly utilize CPU cycles to check if a button is pressed or a sensor is ready. Instead, the system relies heavily on FreeRTOS Binary Semaphores to implement deferred interrupt processing. This approach keeps the Interrupt Service Routines (ISRs) as brief as possible by delegating the actual data processing to the RTOS tasks.

In the case of **Button_Press_Task**, rather than using an endless while loop to check the GPIO state of the push-button, **xSemaphoreTake()** is called and the task enters a blocked state. While blocked, it does not run, allowing other tasks to utilise the CPU. When a user physically presses the button, it triggers a hardware interrupt, briefly pausing normal execution to run the PORTA_IRQHandler. The ISR does not evaluate the button logic itself. It instead clears the hardware interrupt flag, gives the buttonSemaphore via **xSemaphoreGiveFromISR**, and exits. This action instantly unblocks the **Button_Press_Task**, waking it up to debounce the input and queue the UART packet necessary to switch the ESP32's OLED display.

In the case of **Joystick_Control_Task**, parallel synchronisation is applied when reading the analog X/Y axes of the joystick. Analog-to-Digital (ADC) conversions inherently take a few clock cycles to complete, so instead of forcing the CPU to wait idly while the hardware converts the voltage, the task triggers the ADC read and immediately blocks itself using **adcReadySemaphore**. Once the ADC hardware finishes capturing the analog voltage, it triggers **ADC0_IRQHandler**, which gives the

semaphore, waking the **Joystick_Control_Task** once the fresh analog data is ready in the register.

Resource Protection

To prevent potential data corruption due to concurrent access or interleaving of processes, FreeRTOS primitives are used on both devices. On the ESP32, a Mutex (telegramMutex) protects the shared **telegramMsgQueue**. The system is designed such that Core 1 pushes hardware alerts to this queue, while Core 0 pops them to transmit over Wi-Fi. In this manner, the mutex guarantees memory safety by preventing simultaneous core access.

```
// Function to safely push messages to the Telegram Core 0 Task
void queueTelegramMessage(String text, String mode = "") {
  if (telegramMutex != NULL) {
    xSemaphoreTake(telegramMutex, portMAX_DELAY);
    telegramMsgQueue.push({text, mode});
    xSemaphoreGive(telegramMutex);
  }
}
```

Fig 7: Mutex used for Telegram messages

On the MCXC444, FreeRTOS Queues (rxQueue, txQueue) are utilized to safely pass structured data packets between the high-priority UART tasks and the lower-priority user interface tasks, eliminating race conditions.

```
    rxQueue = xQueueCreate(QLEN, sizeof(ReceiveData_t)); // Holds raw bytes from ESP32
    txQueue = xQueueCreate(QLEN, sizeof(SendData_t)); // Holds structs to send to ESP32
```

Fig 8: Queues used for UART related tasks

Communication Protocol

UART Configuration

Full-duplex serial communication between the ESP32 and MCXC444 is established using hardware UART at a baud rate of 9600 bps with 8N1 configuration. In order to maximize CPU efficiency, the MCXC444 utilizes the UART2_FLEXIO_IRQHandler so that transmission and reception can be handled asynchronously. This allows the processor to queue bytes in the background rather than busy-waiting for the serial register to empty.

Packet description

Data is exchanged using strictly defined structs SendData_t and ReceiveData_t. To guarantee that the bytes align perfectly across the two different microcontroller architectures, the structs are defined with the __attribute__((packed)) compiler directive, preventing the insertion of hidden memory padding.

```c
typedef struct __attribute__((packed)) {
    uint8_t startByte; // 0xAA
    bool requestIncreaseLightSensitivty;
    bool requestIncreaseHumiditySensitivity;
    bool requestToggleLED;
    bool requestToggleBuzzer;
    bool requestDisplaySwitch;
} SendData_t;


typedef struct __attribute__((packed)) {
    uint8_t startByte; // 0xAA
    bool triggerLED;
} ReceiveData_t;

```

Fig 9: Defined structs for packets on the MCXC

The protocol also implements robust packet framing. Transmitted structs begin with a designated synchronization byte (0xAA). The receiving UART task is programmed to discard all incoming bytes until this specific sync byte is detected.

SendData_t: This packet transmits user interface commands from the joystick on the MCXC444 to the sensors connected to the ESP32.

Packet Component	Description
uint8_t	0xAA byte used for packet framing.
bool requestIncreaseLightSensitivity	Set to true when the joystick is flicked right. Tell ESP32 to increase currentLightPercentThreshold such that the alarm is triggered without the environment having to be as dark.
bool requestIncreaseHumiditySensitivity	Set to true when the joystick is flicked to the left. Tell ESP32 to increase currentHumidityPercent such that the humidity required to trigger the alarm system is lower than before.
bool requestToggleLED	Set to true when the joystick is flicked downwards, toggle alertWithLED to decide if LED flickers upon unfavourable environmental conditions.
bool requestToggleBuzzer	Set to true when the joystick is flicked upwards, toggles alertWithBuzzer to decide if buzzer beeps upon

	unfavourable environmental conditions.
bool requestDisplaySwitch	Set to true when the joystick button is pressed. Tell ESP32 to update the currentDisplay value to decide which display page to show on the LCD.

ReceiveData_t: This packet serves as an urgent hardware interrupt trigger sent from the sensors on the ESP32 back to the MCXC444.

Packet Component	Description
uint8_t startByte	0xAA synchronisation byte
bool triggerLED	Evaluates to true when the ESP32 detects that an environmental threshold (like rain or high humidity) has been breached, provided the user has the LED alert setting enabled.

Additional Features

The system leverages the ESP32's dual-core architecture using FreeRTOS. We have Core 1 of the ESP32 running the main application loop where It continuously polls the water, temperature & humidity and light sensors, while managing UART communication, and updates the LCD. We then have Core 0 working on anything related to TelegramTask. It is dedicated to handling Wi-Fi network requests and polling the Telegram API for user commands. This way, the network operations can be isolated from the time-sensitive hardware processes.

Online Function Used

To provide remote monitoring and control, the system utilises internet connectivity by exchanging bidirectional data with the Telegram Bot API. Using UniversalTelegramBot.h library and WiFiClientSecure, the ESP32 establishes an encrypted HTTPS connection to Telegram's servers using a nearby WiFi network. This allows for users to remotely check real-time environmental readings and adjust preferred thresholds through their phones while the ESP32 actively pushes alerts when triggered by sensor readings.

The bot continuously polls the Telegram API at 1000ms intervals for new messages. When a new message is sent by the user, the handleNewMessages() function parses the text and executes the corresponding command. To ensure security and

prevent unauthorized manipulation of the IoT device, the incoming payload's Chat ID is verified against a hardcoded user's Chat ID from telegram, ignoring all requests from unregistered accounts.

Challenges faced & Solutions

One of the key challenges encountered during development was the slow and occasionally unresponsive UART communication between the ESP32 and the MCXC444, especially during Wi-Fi and Telegram operations. Initially, the ESP32 executed all tasks within a single loop, causing blocking behaviour where network operations delayed UART handling and sensor updates. To address this, the ESP32 was restructured using FreeRTOS by introducing a dedicated Telegram task via xTaskCreatePinnedToCore(). This enabled concurrent execution, allowing UART and sensor tasks to run independently. Additionally, vTaskDelay() was used to prevent CPU monopolisation, improving responsiveness and overall system stability.

Another challenge was ensuring reliable UART data integrity between the ESP32 and the MCXC444. Since UART is a continuous stream-based protocol, there was a risk of packet misalignment or corrupted data, which could lead to incorrect interpretation of incoming commands. To mitigate this, a framing protocol using a start byte (0xAA) was implemented. The system validates incoming data by checking for this sync byte before attempting to parse the packet into a structured format. In addition, fixed-size packed data structures were used to ensure consistent memory layout across both devices. These measures improved communication reliability by reducing the likelihood of invalid or misaligned UART packet processing.

Conclusion

In conclusion, the system addresses the problem of unpredictable environmental conditions affecting outdoor laundry by providing real-time monitoring and alerts. The ESP32 handles sensor data processing and remote communication, while the MCXC444 provides a responsive user interface using FreeRTOS.

The use of RTOS concepts such as task prioritisation, queues, and semaphores ensures efficient and responsive system behaviour. Additionally, Telegram integration enables convenient remote monitoring and control.